```c
//###########################################################################
// FILE:   FinalProjectStarter_main.c
//
// TITLE:  Final Project Starter
//###########################################################################

// Included Files
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <math.h>
#include <limits.h>
#include "F28x_Project.h"
#include "F2837xD_SWPrioritizedIsrLevels.h"
#include "driverlib.h"
#include "device.h"
#include "F28379dSerial.h"
#include "song.h"
#include "dsp.h"
#include "fpu32/fpu_rfft.h"
#include "xy.h"
#include "MatrixMath.h"
#include "SE423Lib.h"
#include "OptiTrack.h"

#define PI          3.14159265358979323846264338327795
#define TWOPI       6.28318530717958647692528676655 9
#define HALFPI      1.57079632679489661923132169163 98
// The Launchpad's CPU Frequency set to 200 you should not change this value
#define LAUNCHPAD_CPU_FREQUENCY 200

// ----- code for CAN start here -----
#include "F28379dCAN.h"
//#define TX_MSG_DATA_LENGTH    4
//#define TX_MSG_OBJ_ID         0  //transmit

#define RX_MSG_DATA_LENGTH    8
#define RX_MSG_OBJ_ID_1       1  //measurement from sensor 1
#define RX_MSG_OBJ_ID_2       2  //measurement from sensor 2
#define RX_MSG_OBJ_ID_3       3  //quality from sensor 1
#define RX_MSG_OBJ_ID_4       4  //quality from sensor 2
// ----- code for CAN end here -----

// Interrupt Service Routines predefinition
__interrupt void cpu_timer0_isr(void);
__interrupt void cpu_timer1_isr(void);
__interrupt void cpu_timer2_isr(void);
__interrupt void SWI1_HighestPriority(void);
__interrupt void SWI2_MiddlePriority(void);
__interrupt void SWI3_LowestPriority(void);
__interrupt void ADCC_ISR(void);
__interrupt void SPIB_isr(void);
// ----- code for CAN start here -----
__interrupt void can_isr(void);
// ----- code for CAN end here -----

void setF28027EPWM1A(float controleffort);
int16_t EPwm1A_F28027 = 1500;
void setF28027EPWM2A(float controleffort);
int16_t EPwm2A_F28027 = 1500;

// ----- code for CAN start here -----
// volatile uint32_t txMsgCount = 0;
// extern uint16_t txMsgData[4];

volatile uint32_t rxMsgCount_1 = 0;
volatile uint32_t rxMsgCount_2 = 0;
extern uint16_t rxMsgData[8];

uint32_t dis_raw_1[2];
uint32_t dis_raw_2[2];
uint32_t dis_1 = 0;
uint32_t dis_2 = 0;

uint32_t quality_raw_1[4];
uint32_t quality_raw_2[4];
float quality_1 = 0.0;
float quality_2 = 0.0;

uint32_t lightlevel_raw_1[4];
uint32_t lightlevel_raw_2[4];
float lightlevel_1 = 0.0;
float lightlevel_2 = 0.0;

uint32_t measure_status_1 = 0;
uint32_t measure_status_2 = 0;

volatile uint32_t errorFlag = 0;
// ----- code for CAN end here -----

uint32_t numTimer0calls = 0;
uint16_t UARTPrint = 0;

float printLV1 = 0;
float printLV2 = 0;

float printLinux1 = 0;
float printLinux2 = 0;

uint16_t LADARi = 0;
char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
uint16_t G_len = 11; //length of command
xy ladar_pts[228]; //xy data
float LADARrightfront = 0;
float LADARfront = 0;
float LADARtemp_x = 0;
float LADARtemp_y = 0;
extern datapts ladar_data[228];

extern uint16_t newLinuxCommands;
extern float LinuxCommands[CMDNUM_FROM_FLOATS];
```

```
113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1;  // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2;  // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1;  // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 uint32_t numThres1 = 0;
161 uint32_t numThres2 = 0;
162
163 pose ROBOTps = {0,0,0}; //robot position
164 pose LADARps = {3.5/12.0,0,1};  // 3.5/12 for front mounting, theta is not used in this current code
165 float LADARxoffset = 0;
166 float LADARyoffset = 0;
167
168 uint32_t timecount = 0;
169 int16_t RobotState = 1;
170 int16_t checkfronttally = 0;
171 int32_t WallFollowtime = 0;
172
173 #define NUMWAYPOINTS 8
174 uint16_t statePos = 0;
175 pose robotdest[NUMWAYPOINTS];  // array of waypoints for the robot
176 uint16_t i = 0;//for loop
177
178 uint16_t right_wall_follow_state = 2;  // right follow
179 float Kp_front_wall = -2.0;
180 float front_turn_velocity = 0.2;
181 float left_turn_Stop_threshold = 3.5;
182 float Kp_right_wal = -4.0;
183 float ref_right_wall = 1.1;
184 float foward_velocity = 1.0;
185 float left_turn_Start_threshold = 1.3;
186 float turn_saturation = 2.5;
187
188 float x_pred[3][1] = {{0},{0},{0}};                 // predicted state
189
190 //more kalman vars
191 float B[3][2] = {{1,0},{1,0},{0,1}};            // control input model
192 float u[2][1] = {{0},{0}};          // control input in terms of velocity and angular velocity
193 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
194 float z[3][1];                          // state measurement
195 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}};   // 3x3 identity matrix
196 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}};      // optimal Kalman gain
197 #define ProcUncert 0.0001
198 #define CovScalar 10
199 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
200                  {0,ProcUncert,ProcUncert/CovScalar},
201                  {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}};   // process noise (covariance of encoders and gyro)
202 #define MeasUncert 1
203 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
204                  {0,MeasUncert,MeasUncert/CovScalar},
205                  {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}};   // measurement noise (covariance of OptiTrack Motion Capture measurement)
206 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}};  // innovation covariance
207 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}};  // innovation covariance matrix inverse
208 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
209 float temp_3x3[3][3];               // intermediate storage
210 float temp_3x1[3][1];               // intermediate storage
211 float ytilde[3][1];                 // difference between predictions
212
213 // Optitrack Variables
214 int32_t OptiNumUpdatesProcessed = 0;
215 pose OPTITRACKps;
216 extern uint16_t new_optitrack;
217 extern float Optitrackdata[OPTITRACKDATASIZE];
218 int16_t newOPTITRACKpose=0;
219
220 int16_t adcc2result = 0;
221 int16_t adcc3result = 0;
222 int16_t adcc4result = 0;
223 int16_t adcc5result = 0;
224 float adcC2Volt = 0.0;
225 float adcC3Volt = 0.0;
226 float adcC4Volt = 0.0;
```

```
float adcC5Volt = 0.0;
int32_t numADCCcalls = 0;
float LeftWheel = 0;
float RightWheel = 0;
float LeftWheel_1 = 0;
float RightWheel_1 = 0;
float LeftVel = 0;
float RightVel = 0;
float uLeft = 0;
float uRight = 0;
float HandValue = 0;
float vref = 0;
float turn = 0;
float gyro9250_drift = 0;
float gyro9250_angle = 0;
float old_gyro9250 = 0;
float gyroLPR510_angle = 0;
float gyroLPR510_offset = 0;
float gyroLPR510_drift = 0;
float old_gyroLPR510 = 0;
float gyro9250_radians = 0;
float gyroLPR510_radians = 0;

int16_t readdata[25];
int16_t IMU_data[9];

float accelx = 0;
float accely = 0;
float accelz = 0;
float gyrox = 0;
float gyroy = 0;
float gyroz = 0;

// Needed global Variables
float accelx_offset = 0;
float accely_offset = 0;
float accelz_offset = 0;
float gyrox_offset = 0;
float gyroy_offset = 0;
float gyroz_offset = 0;
int16_t calibration_state = 0;
int32_t calibration_count = 0;
int16_t doneCal = 0;

#define MPU9250 1
#define DAN28027 2
int16_t CurrentChip = MPU9250;
int16_t DAN28027Garbage = 0;
int16_t dan28027adc1 = 0;
int16_t dan28027adc2 = 0;
uint16_t MPU9250ignoreCNT = 0;  //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.

void main(void)
{
    // PLL, WatchDog, enable Peripheral Clocks
    // This example function is found in the F2837xD_SysCtrl.c file.
    InitSysCtrl();

    InitGpio();

    InitSE423DefaultGPIO();

    //Scope for Timing
    GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;

    GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;

    GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;

    GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;

    GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;

    GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;

// ----- code for CAN start here -----
    //GPIO17 - CANRXB
    GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
    GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);

    //GPIO12 - CANTXB
    GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
    GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
// ----- code for CAN end here -----



// ----- code for CAN start here -----
    // Initialize the CAN controller
    InitCANB();

    // Set up the CAN bus bit rate to 1000 kbps
    setCANBitRate(200000000, 1000000);

    // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
    CanbRegs.CAN_CTL.bit.IE0= 1;
    CanbRegs.CAN_CTL.bit.EIE= 1;
// ----- code for CAN end here -----

    // Clear all interrupts and initialize PIE vector table:
    // Disable CPU interrupts
    DINT;
```

```
341
342     // Initialize the PIE control registers to their default state.
343     // The default state is all PIE interrupts disabled and flags
344     // are cleared.
345     // This function is found in the F2837xD_PieCtrl.c file.
346     InitPieCtrl();
347
348     // Disable CPU interrupts and clear all CPU interrupt flags:
349     IER = 0x0000;
350     IFR = 0x0000;
351
352     // Initialize the PIE vector table with pointers to the shell Interrupt
353     // Service Routines (ISR).
354     // This will populate the entire table, even if the interrupt
355     // is not used in this example.  This is useful for debug purposes.
356     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
357     // This function is found in F2837xD_PieVect.c.
358     InitPieVectTable();
359
360     // Interrupts that are used in this example are re-mapped to
361     // ISR functions found within this project
362     EALLOW;  // This is needed to write to EALLOW protected registers
363     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
364     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
365     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
366     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
367     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
368     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
369     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
370     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
371     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
372     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
373     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
374     PieVectTable.ADCC1_INT = &ADCC_ISR;
375     PieVectTable.SPIB_RX_INT = &SPIB_isr;
376     PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority;  // Using Interrupt12 interrupts that are not used as SWIs
377     PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
378     PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
379         // ----- code for CAN start here -----
380     PieVectTable.CANB0_INT = &can_isr;
381 // ----- code for CAN end here -----
382     EDIS;    // This is needed to disable write to EALLOW protected registers
383
384     // Initialize the CpuTimers Device Peripheral. This function can be
385     // found in F2837xD_CpuTimers.c
386     InitCpuTimers();
387
388     // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
389     // 200MHz CPU Freq,                     Period (in uSeconds)
390     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);  // Currently not used for any purpose
391     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms.  Do not Change.
392     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
393
394     // Enable CpuTimer Interrupt bit TIE
395     CpuTimer0Regs.TCR.all = 0x4000;
396     CpuTimer1Regs.TCR.all = 0x4000;
397     CpuTimer2Regs.TCR.all = 0x4000;
398
399     DELAY_US(1000000);  // Delay 1 second giving Lidar Time to power on after system power on
400
401     init_serialSCIB(&SerialB,19200);
402     init_serialSCIC(&SerialC,19200);
403
404     for (LADARi = 0; LADARi < 228; LADARi++) {
405         ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
406     }
407
408     init_eQEPs();
409     init_EPWM1and2();
410     init_RCServoPWM_3AB_5AB_6A();
411     init_ADCsAndDACs();
412     setupSpib();
413
414     EALLOW;
415     EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
416     EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
417     EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
418     EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
419     EPwm4Regs.TBCTR = 0x0; // Clear counter
420     EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
421     EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
422     EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1  50Mhz Clock
423     EPwm4Regs.TBPRD = 50000;  // Set Period to 1ms sample.  Input clock is 50MHz.
424     // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
425     EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
426     //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze,  wait to do this right before enabling interrupts
427     EDIS;
428
429     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
430     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
431     // to CPU-Timer 2:  int 12 is for the SWI.
432     IER |= M_INT1;
433     IER |= M_INT6;
434     IER |= M_INT8;  // SCIC SCID
435     IER |= M_INT9;  // SCIA
436     IER |= M_INT12;
437     IER |= M_INT13;
438     IER |= M_INT14;
439
440     // Enable TINT0 in the PIE: Group 1 interrupt 7
441     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
442     PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
443     PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
444     PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
445     PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
446     PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3  Lowest priority
447         // ----- code for CAN start here -----
448     // Enable CANB in the PIE: Group 9 interrupt 7
449     PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
450 // ----- code for CAN end here -----
451
452 // ----- code for CAN start here -----
453     // Enable the CAN interrupt signal
454     CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
```

```
// ----- code for CAN end here -----


    robotdest[0].x = -4;    robotdest[0].y = 10;
    robotdest[1].x = -4;    robotdest[1].y = 2;
    //middle of bottom
    robotdest[2].x = 0;     robotdest[2].y = 2;
    //outside the course
    robotdest[3].x = 0;     robotdest[3].y = -3;
    //back to middle
    robotdest[4].x = 0;     robotdest[4].y = 2;
    robotdest[5].x = 4;     robotdest[5].y = 2;
    robotdest[6].x = 4;     robotdest[6].y = 10;
    robotdest[7].x = 0;     robotdest[7].y = 9;

    // ROBOTps will be updated by Optitrack during gyro calibration
    // TODO: specify the starting position of the robot
    ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
    ROBOTps.y = 0;
    ROBOTps.theta = 0;  // was -PI: need to flip OT ground plane to fix this
    x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
    x_pred[1][0] = ROBOTps.y;
    x_pred[2][0] = ROBOTps.theta;

    AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;  //clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
    SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
    SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
    EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode

    init_serialSCIA(&SerialA,115200);
    init_serialSCID(&SerialD,2083332);

    // Enable global Interrupts and higher priority real-time debug events
    EINT;  // Enable Global interrupt INTM
    ERTM;  // Enable Global realtime interrupt DBGM
// ----- code for CAN start here -----
    // Measured Distance from 1
    // Initialize the receive message object 1 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 1
    //      Message Identifier: 0x060b0101
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //
    CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
                          CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
                          RX_MSG_DATA_LENGTH);

    // Measured Distance from 2
    // Initialize the receive message object 2 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 2
    //      Message Identifier: 0x060b0102
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //

    CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
                          CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
                          RX_MSG_DATA_LENGTH);

    // Measurement Quality from 1
    // Initialize the receive message object 2 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 3
    //      Message Identifier: 0x060b0201
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //

    CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
                          CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
                          RX_MSG_DATA_LENGTH);

    // Measurement Quality from 2
    // Initialize the receive message object 2 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 4
    //      Message Identifier: 0x060b0202
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //

    CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
                          CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
                          RX_MSG_DATA_LENGTH);

    //
    // Start CAN module operations
    //
    CanbRegs.CAN_CTL.bit.Init = 0;
    CanbRegs.CAN_CTL.bit.CCE = 0;


// ----- code for CAN end here -----
    char S_command[19] = "S1152000124000\n";//this change the baud rate to 115200
```

```
    uint16_t S_len = 19;
    serial_sendSCIC(&SerialC, S_command, S_len);


    DELAY_US(1000000);  // Delay letting Lidar change its Baud rate
    init_serialSCIC(&SerialC,115200);


    // LED1 is GPIO22
    GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
    // LED2 is GPIO94
    GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
    // LED3 is GPIO95
    GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
    // LED4 is GPIO97
    GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
    // LED5 is GPIO111
    GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;


    // IDLE loop. Just sit and loop forever (optional):
    while(1)
    {
        if (UARTPrint == 1 ) {

            if (readbuttons() == 0) {
                UART_printfLine(1,"Vrf:%.2f trn:%.2f",vref,turn);
//                UART_printfLine(1,"x:%.2f:y:%.2f:a%.2f",ROBOTps.x,ROBOTps.y,ROBOTps.theta);
                UART_printfLine(2,"F%.4f R%.4f",LADARfront,LADARrightfront);
            } else if (readbuttons() == 1) {
                UART_printfLine(1,"O1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold1,MaxColThreshold1,MaxRowThreshold1);
                UART_printfLine(2,"P1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold2,MaxColThreshold2,MaxRowThreshold2);
                            //UART_printfLine(1,"LV1:%.3f LV2:%.3f",printLV1,printLV2);
                //UART_printfLine(2,"Ln1:%.3f Ln2:%.3f",printLinux1,printLinux2);
            } else if (readbuttons() == 2) {
                UART_printfLine(1,"O2A:%.0fC:%.0fR:%.0f",NextLargestAreaThreshold1,NextLargestColThreshold1,NextLargestRowThreshold1);
                UART_printfLine(2,"P2A:%.0fC:%.0fR:%.0f",NextLargestAreaThreshold2,NextLargestColThreshold2,NextLargestRowThreshold2);
                // UART_printfLine(1,"%.2f %.2f",adcC2Volt,adcC3Volt);
                // UART_printfLine(2,"%.2f %.2f",adcC4Volt,adcC5Volt);
            } else if (readbuttons() == 4) {
                UART_printfLine(1,"O3A:%.0fC:%.0fR:%.0f",NextNextLargestAreaThreshold1,NextNextLargestColThreshold1,NextNextLargestRowThreshold1);
                UART_printfLine(2,"P3A:%.0fC:%.0fR:%.0f",NextNextLargestAreaThreshold2,NextNextLargestColThreshold2,NextNextLargestRowThreshold2);
                // UART_printfLine(1,"L:%.3f R:%.3f",LeftVel,RightVel);
                // UART_printfLine(2,"uL:%.2f uR:%.2f",uLeft,uRight);
            } else if (readbuttons() == 8) {
                UART_printfLine(1,"020x%.2f y%.2f",ladar_pts[20].x,ladar_pts[20].y);
                UART_printfLine(2,"150x%.2f y%.2f",ladar_pts[150].x,ladar_pts[150].y);
            } else if (readbuttons() == 3) {
                UART_printfLine(1,"Vrf:%.2f trn:%.2f",vref,turn);
                UART_printfLine(2,"MPU:%.2f LPR:%.2f",gyro9250_radians,gyroLPR510_radians);
            } else if (readbuttons() == 5) {
                UART_printfLine(1,"Ox:%.2f:Oy:%.2f:Oa%.2f",OPTITRACKps.x,OPTITRACKps.y,OPTITRACKps.theta);
                UART_printfLine(2,"State:%d : %d",RobotState,statePos);
                        } else if (readbuttons() == 6) {
                                UART_printfLine(1,"D1 %ld D2 %ld",dis_1,dis_2);
                UART_printfLine(2,"St1 %ld St2 %ld",measure_status_1,measure_status_2);
            } else if (readbuttons() == 7) {
                UART_printfLine(1,"%.0f,%.1f,%.1f,%.1f",tagid,tagx,tagy,tagz);
                UART_printfLine(2,"%.1f,%.1f,%.1f",tagthetax,tagthetay,tagthetaz);
            }

            UARTPrint = 0;
        }
    }
}

__interrupt void SPIB_isr(void){

    uint16_t i;
    GpioDataRegs.GPBSET.bit.GPIO32 = 1;

    GpioDataRegs.GPCSET.bit.GPIO66 = 1;                 //Pull CS high as done R/W
    GpioDataRegs.GPASET.bit.GPIO29 = 1;  // Pull CS high for DAN28027

    if (CurrentChip == MPU9250) {

        for (i=0; i<8; i++) {
            readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
        }

        PostSWI1(); // Manually cause the interrupt for the SWI1

    } else if (CurrentChip == DAN28027) {

        DAN28027Garbage = SpibRegs.SPIRXBUF;
        dan28027adc1 = SpibRegs.SPIRXBUF;
        dan28027adc2 = SpibRegs.SPIRXBUF;
        CurrentChip = MPU9250;
        SpibRegs.SPIFFCT.bit.TXDLY = 0;
    }

    SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
    SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
    GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
}


//adcd1 pie interrupt
__interrupt void ADCC_ISR (void)
{
    GpioDataRegs.GPASET.bit.GPIO11 = 1;
    adcc2result = AdccResultRegs.ADCRESULT0;
    adcc3result = AdccResultRegs.ADCRESULT1;
    adcc4result = AdccResultRegs.ADCRESULT2;
    adcc5result = AdccResultRegs.ADCRESULT3;

    // Here covert ADCIND0 to volts
    adcC2Volt = adcc2result*3.0/4095.0;
    adcC3Volt = adcc3result*3.0/4095.0;
    adcC4Volt = adcc4result*3.0/4095.0;
    adcC5Volt = adcc5result*3.0/4095.0;

    if (MPU9250ignoreCNT >= 1) {
```

```
        CurrentChip = MPU9250;
        SpibRegs.SPIFFCT.bit.TXDLY = 0;
        SpibRegs.SPIFFRX.bit.RXFFIL = 8;

        GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;

        SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
    } else {
        MPU9250ignoreCNT++;
    }

    numADCCcalls++;
    AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;  //clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
    GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;

}

// cpu_timer0_isr - CPU Timer0 ISR
__interrupt void cpu_timer0_isr(void)
{
    CpuTimer0.InterruptCount++;

    numTimer0calls++;

    if ((numTimer0calls%5) == 0) {
        // Blink LaunchPad Red LED
        //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
    }

    // Acknowledge this interrupt to receive more interrupts from group 1
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}

// cpu_timer1_isr - CPU Timer1 ISR
__interrupt void cpu_timer1_isr(void)
{

    serial_sendSCIC(&SerialC, G_command, G_len);

    CpuTimer1.InterruptCount++;
}

// cpu_timer2_isr CPU Timer2 ISR
__interrupt void cpu_timer2_isr(void)
{
    // Blink LaunchPad Blue LED
    //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;

    CpuTimer2.InterruptCount++;

    //  if ((CpuTimer2.InterruptCount % 10) == 0) {
    //      UARTPrint = 1;
    //  }
}

void setF28027EPWM1A(float controleffort){
    if (controleffort < -10) {
        controleffort = -10;
    }
    if (controleffort > 10) {
        controleffort = 10;
    }
    float value = (controleffort+10)*3000.0/20.0;
    EPwm1A_F28027 = (int16_t)value;  // Set global variable that is sent over SPI to F28027
}
void setF28027EPWM2A(float controleffort){
    if (controleffort < -10) {
        controleffort = -10;
    }
    if (controleffort > 10) {
        controleffort = 10;
    }
    float value = (controleffort+10)*3000.0/20.0;
    EPwm2A_F28027 = (int16_t)value;  // Set global variable that is sent over SPI to F28027
}

//
// Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
//
__interrupt void SWI1_HighestPriority(void)     // EMIF_ERROR
{
    GpioDataRegs.GPBSET.bit.GPIO61 = 1;
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER    &= MINT12;                           // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_9;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
    __asm("  NOP");
    EINT;

    //#############################################################################################################
    IMU_data[0] = readdata[1];
    IMU_data[1] = readdata[2];
    IMU_data[2] = readdata[3];
    IMU_data[3] = readdata[5];
    IMU_data[4] = readdata[6];
    IMU_data[5] = readdata[7];

    accelx = (((float)(IMU_data[0]))*4.0/32767.0);
    accely = (((float)(IMU_data[1]))*4.0/32767.0);
    accelz = (((float)(IMU_data[2]))*4.0/32767.0);
    gyrox  = (((float)(IMU_data[3]))*250.0/32767.0);
    gyroy  = (((float)(IMU_data[4]))*250.0/32767.0);
    gyroz  = (((float)(IMU_data[5]))*250.0/32767.0);

```

```
    if(calibration_state == 0){
        calibration_count++;
        if (calibration_count == 2000) {
            calibration_state = 1;
            calibration_count = 0;
        }
    } else if(calibration_state == 1){
        accelx_offset+=accelx;
        accely_offset+=accely;
        accelz_offset+=accelz;
        gyrox_offset+=gyrox;
        gyroy_offset+=gyroy;
        gyroz_offset+=gyroz;
        gyroLPR510_offset+=adcC5Volt;
        calibration_count++;
        if (calibration_count == 2000) {
            calibration_state = 2;
            accelx_offset/=2000.0;
            accely_offset/=2000.0;
            accelz_offset/=2000.0;
            gyrox_offset/=2000.0;
            gyroy_offset/=2000.0;
            gyroz_offset/=2000.0;
            gyroLPR510_offset/=2000.0;
            calibration_count = 0;
        }
    } else if(calibration_state == 2) {

        gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
        old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;

        gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
        old_gyro9250 = gyroz-gyroz_offset;
        calibration_count++;

        if (calibration_count == 2000) {
            calibration_state = 3;
            calibration_count = 0;
            doneCal = 1;
            newOPTITRACKpose = 0;
        }
    } else if(calibration_state == 3){
        accelx -=(accelx_offset);
        accely -=(accely_offset);
        accelz -=(accelz_offset);
        gyrox -= gyrox_offset;
        gyroy -= gyroy_offset;
        gyroz -= gyroz_offset;
        LeftWheel = readEncLeft();
        RightWheel = readEncRight();
        HandValue = readEncWheel();

        gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
        old_gyro9250 = gyroz;

        gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
        old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;

        gyro9250_radians = (gyro9250_angle * (PI/180.0));
        gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);

        LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
        RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;

        if (newLinuxCommands == 1) {
            newLinuxCommands = 0;
            printLinux1 = LinuxCommands[0];
            printLinux2 = LinuxCommands[1];
            //?? = LinuxCommands[2];
            //?? = LinuxCommands[3];
            //?? = LinuxCommands[4];
            //?? = LinuxCommands[5];
            //?? = LinuxCommands[6];
            //?? = LinuxCommands[7];
            //?? = LinuxCommands[8];
            //?? = LinuxCommands[9];
            //?? = LinuxCommands[10];
        }

        if (NewCAMDataThreshold1 == 1) {
            NewCAMDataThreshold1 = 0;
            MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
            MaxColThreshold1 = fromCAMvaluesThreshold1[1];
            MaxRowThreshold1 = fromCAMvaluesThreshold1[2];

            NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
            NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
            NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];

            NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
            NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
            NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
                        numThres1++;
            if ((numThres1 % 5) == 0) {
                // LED4 is GPIO97
                GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
            }
        }

        if (NewCAMDataThreshold2 == 1) {
            NewCAMDataThreshold2 = 0;
            MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
            MaxColThreshold2 = fromCAMvaluesThreshold2[1];
            MaxRowThreshold2 = fromCAMvaluesThreshold2[2];

            NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
            NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
            NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];

            NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
            NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
            NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
                        numThres2++;
                        if ((numThres2 % 5) == 0) {
```

```
            // LED5 is GPIO111
            GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
        }
    }

    if (NewCAMDataAprilTag1 == 1) {
        NewCAMDataAprilTag1 = 0;
        tagx = fromCAMvaluesAprilTag1[0];
        tagy = fromCAMvaluesAprilTag1[1];
        tagz = fromCAMvaluesAprilTag1[2];

        tagthetax = fromCAMvaluesAprilTag1[3];
        tagthetay = fromCAMvaluesAprilTag1[4];
        tagthetaz = fromCAMvaluesAprilTag1[5];

        tagid = fromCAMvaluesAprilTag1[6];

        numtag++;
        if ((numtag % 5) == 0) {
            // LED2 is GPIO94
            GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
        }
    }

    if (NewLVData == 1) {
        NewLVData = 0;
        printLV1 = fromLVvalues[0];
        printLV2 = fromLVvalues[1];
        //?? = fromLVvalues[2];
        //?? = fromLVvalues[3];
        //?? = fromLVvalues[4];
        //?? = fromLVvalues[5];
        //?? = fromLVvalues[6];
        //?? = fromLVvalues[7];
    }


    if (new_optitrack == 1) {
        OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
        new_optitrack = 0;
    }

    // Step 0: update B, u
    B[0][0] = cosf(ROBOTps.theta)*0.001;
    B[1][0] = sinf(ROBOTps.theta)*0.001;
    B[2][1] = 0.001;
    u[0][0] = 0.5*(LeftVel + RightVel);    // linear velocity of robot
    u[1][0] = gyroz*(PI/180.0);    // angular velocity in rad/s (negative for right hand angle)

    // Step 1: predict the state and estimate covariance
    Matrix3x2_Mult(B, u, Bu);                   // Bu = B*u
    Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
    Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
    // Step 2: if there is a new measurement, then update the state
    if (1 == newOPTITRACKpose) {
        OptiNumUpdatesProcessed++;  // checking how often OptiTrack is causing a Kalman update
        if ((OptiNumUpdatesProcessed%10)==0) {
            // LED 3
            GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
        }
        newOPTITRACKpose = 0;
        z[0][0] = OPTITRACKps.x;    // take in the Optitrack Motion Capture measurement
        z[1][0] = OPTITRACKps.y;
        // fix for OptiTrack problem at 180 degrees
        if (cosf(ROBOTps.theta) < -0.99) {
            z[2][0] = ROBOTps.theta;
        }
        else {
            z[2][0] = OPTITRACKps.theta;
        }
        // Step 2a: calculate the innovation/measurement residual, ytilde
        Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0);    // ytilde = z-x_pred
        // Step 2b: calculate innovation covariance, S
        Matrix3x3_Add(P_pred, R, S, 1.0, 1.0);                          // S = P_pred + R
        // Step 2c: calculate the optimal Kalman gain, K
        Matrix3x3_Invert(S, S_inv);
        Matrix3x3_Mult(P_pred,  S_inv, K);                              // K = P_pred*(S^-1)
        // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
        Matrix3x1_Mult(K, ytilde, temp_3x1);
        Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
        // Step 2e: update the covariance estimate   P_pred = (I-K)*P_pred(old)
        Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
        Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
    }   // end of correction step

    // set ROBOTps to the updated and corrected Kalman values.
    ROBOTps.x = x_pred[0][0];
    ROBOTps.y = x_pred[1][0];
    ROBOTps.theta = x_pred[2][0];

    // Make sure this function is called every time in this function even if you decide not to use its vref and turn
    // uses xy code to step through an array of positions
    if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
        statePos = (statePos+1)%NUMWAYPOINTS;
    }
    // state machine
    switch (RobotState) {
    case 1:

        // vref and turn are the vref and turn returned from xy_control

        if (LADARfront < 1.2) {
            vref = 0.2;
            checkfronttally++;
            if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
                RobotState = 10; // Wall follow
                WallFollowtime = 0;
                right_wall_follow_state = 1;
            }
        } else {
            checkfronttally = 0;
        }

        break;
```

```
        case 10:
            if (right_wall_follow_state == 1) {
                //Left Turn
                turn = Kp_front_wall*(14.5 - LADARfront);
                vref = front_turn_velocity;
                if (LADARfront > left_turn_Stop_threshold) {
                    right_wall_follow_state = 2;
                }
            } else if (right_wall_follow_state == 2) {
                //Right Wall Follow
                turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
                vref = foward_velocity;
                if (LADARfront < left_turn_Start_threshold) {
                    right_wall_follow_state = 1;
                }
            }
            if (turn > turn_saturation) {
                turn = turn_saturation;
            }
            if (turn < -turn_saturation) {
                turn = -turn_saturation;
            }

            WallFollowtime++;
            if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
                RobotState = 1;
                checkfronttally = 0;
            }
            break;

        case 20:
            // put vision code here
            break;
        default:
            break;
        }


        //Must be called each time into this SWI1 function
        PIcontrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
        // These below lines also must be called each time into this SWI1 function
        setEPWM1A(uLeft);
        setEPWM2A(-uRight);
        setF28027EPWM1A(uLeft);
        setF28027EPWM2A(-uRight);
        LeftWheel_1 = LeftWheel;
        RightWheel_1 = RightWheel;

        if((timecount%250) == 0) {
            DataToLabView.floatData[0] = ROBOTps.x;
            DataToLabView.floatData[1] = ROBOTps.y;
            DataToLabView.floatData[2] = ROBOTps.theta;
            DataToLabView.floatData[3] = (float)timecount;
            DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
            DataToLabView.floatData[5] = (float)RobotState;
            DataToLabView.floatData[6] = (float)statePos;
            DataToLabView.floatData[7] = LADARfront;
            LVsenddata[0] = '*';  // header for LVdata
            LVsenddata[1] = '$';
            for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
                if (i%2==0) {
                    LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
                } else {
                    LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
                }
            }
            serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
        }

    }
    timecount++;
    if((timecount%200) == 0)
    {
        if(doneCal == 0) {
            GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
        }
        GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
        UARTPrint = 1; // Tell While loop to print
    }

    SpibRegs.SPIFFCT.bit.TXDLY = 16;
    CurrentChip = DAN28027;
    SpibRegs.SPIFFRX.bit.RXFFIL = 3;
    GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
    SpibRegs.SPITXBUF = 0x00DA;
    SpibRegs.SPITXBUF = EPwm1A_F28027;
    SpibRegs.SPITXBUF = EPwm2A_F28027;


    //###########################################################################################################
    //
    // Restore registers saved:
    //
    DINT;
    PieCtrlRegs.PIEIER12.all = TempPIEIER;

    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
}

//
// Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
//
__interrupt void SWI2_MiddlePriority(void)     // RAM_CORRECTABLE_ERROR
{
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER    &= MINT12;                           // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_10;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
    __asm("  NOP");
    EINT;

    //###########################################################################################################
```

```
    // Insert SWI ISR Code here.......
    // LED1
    GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
    if (LADARpingpong == 1) {
        // LADARrightfront is the min of dist 52, 53, 54, 55, 56
        LADARrightfront = 19; // 19 is greater than max feet
        for (LADARi = 52; LADARi <= 56 ; LADARi++) {
            if (ladar_data[LADARi].distance_ping < LADARrightfront) {
                LADARrightfront = ladar_data[LADARi].distance_ping;
            }
        }
        // LADARfront is the min of dist 111, 112, 113, 114, 115
        LADARfront = 19;
        for (LADARi = 111; LADARi <= 115 ; LADARi++) {
            if (ladar_data[LADARi].distance_ping < LADARfront) {
                LADARfront = ladar_data[LADARi].distance_ping;
            }
        }
        LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
        LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
        for (LADARi = 0; LADARi < 228; LADARi++) {

            ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
            ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);

        }
    } else if (LADARpingpong == 0) {
        // LADARrightfront is the min of dist 52, 53, 54, 55, 56
        LADARrightfront = 19; // 19 is greater than max feet
        for (LADARi = 52; LADARi <= 56 ; LADARi++) {
            if (ladar_data[LADARi].distance_pong < LADARrightfront) {
                LADARrightfront = ladar_data[LADARi].distance_pong;
            }
        }
        // LADARfront is the min of dist 111, 112, 113, 114, 115
        LADARfront = 19;
        for (LADARi = 111; LADARi <= 115 ; LADARi++) {
            if (ladar_data[LADARi].distance_pong < LADARfront) {
                LADARfront = ladar_data[LADARi].distance_pong;
            }
        }
        LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
        LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
        for (LADARi = 0; LADARi < 228; LADARi++) {

            ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
            ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);

        }
    }




    //###########################################################################################
    //
    // Restore registers saved:
    //
    DINT;
    PieCtrlRegs.PIEIER12.all = TempPIEIER;

}

//
// Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
//
__interrupt void SWI3_LowestPriority(void)     // FLASH_CORRECTABLE_ERROR
{
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER    &= MINT12;                          // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_11;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
    __asm("  NOP");
    EINT;

    //###########################################################################################
    // Insert SWI ISR Code here.......

    //###########################################################################################
    //
    // Restore registers saved:
    //
    DINT;
    PieCtrlRegs.PIEIER12.all = TempPIEIER;

}

// ----- code for CAN start here -----
__interrupt void can_isr(void)
{
    int i = 0;

    uint32_t status;

    //
    // Read the CAN interrupt status to find the cause of the interrupt
    //
    status = CANgetInterruptCause(CANB_BASE);

    //
    // If the cause is a controller status interrupt, then get the status
    //
    if(status == CAN_INT_INT0ID_STATUS)
    {
        //
        // Read the controller status.  This will return a field of status
        // error bits that can indicate various errors.  Error processing
        // is not done in this example for simplicity.  Refer to the
        // API documentation for details about the error status bits.
        // The act of reading this status will clear the interrupt.
        //
        status = CANgetStatus(CANB_BASE);
```

```
1253
1254    }
1255
1256    //
1257    // Check if the cause is the receive message object 2
1258    //
1259    else if(status == RX_MSG_OBJ_ID_1)
1260    {
1261        //
1262        // Get the received message
1263        //
1264        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1265
1266        for(i = 0; i<2; i++)
1267        {
1268            dis_raw_1[i] = rxMsgData[i];
1269        }
1270
1271        dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1272
1273        measure_status_1 = rxMsgData[2];
1274
1275        //
1276        // Getting to this point means that the RX interrupt occurred on
1277        // message object 2, and the message RX is complete.  Clear the
1278        // message object interrupt.
1279        //
1280        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1281
1282        //
1283        // Increment a counter to keep track of how many messages have been
1284        // received. In a real application this could be used to set flags to
1285        // indicate when a message is received.
1286        //
1287        rxMsgCount_1++;
1288
1289        //
1290        // Since the message was received, clear any error flags.
1291        //
1292        errorFlag = 0;
1293    }
1294
1295    else if(status == RX_MSG_OBJ_ID_2)
1296    {
1297        //
1298        // Get the received message
1299        //
1300        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1301
1302        for(i = 0; i<2; i++)
1303        {
1304            dis_raw_2[i] = rxMsgData[i];
1305        }
1306
1307        dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1308
1309        measure_status_2 = rxMsgData[2];
1310
1311        //
1312        // Getting to this point means that the RX interrupt occurred on
1313        // message object 2, and the message RX is complete.  Clear the
1314        // message object interrupt.
1315        //
1316        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1317
1318        //
1319        // Increment a counter to keep track of how many messages have been
1320        // received. In a real application this could be used to set flags to
1321        // indicate when a message is received.
1322        //
1323        rxMsgCount_2++;
1324
1325        //
1326        // Since the message was received, clear any error flags.
1327        //
1328        errorFlag = 0;
1329    }
1330
1331    else if(status == RX_MSG_OBJ_ID_3)
1332    {
1333        //
1334        // Get the received message
1335        //
1336        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1337
1338        for(i = 0; i<4; i++)
1339        {
1340            lightlevel_raw_1[i] = rxMsgData[i];
1341            quality_raw_1[i] = rxMsgData[i+4];
1342
1343        }
1344
1345        lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1346        quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1347
1348
1349        //
1350        // Getting to this point means that the RX interrupt occurred on
1351        // message object 2, and the message RX is complete.  Clear the
1352        // message object interrupt.
1353        //
1354        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1355
1356        //
1357        // Since the message was received, clear any error flags.
1358        //
1359        errorFlag = 0;
1360    }
1361
1362
1363    else if(status == RX_MSG_OBJ_ID_4)
1364    {
1365        //
1366        // Get the received message
```

```
1367         //
1368         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1369
1370         for(i = 0; i<4; i++)
1371         {
1372             lightlevel_raw_2[i] = rxMsgData[i];
1373             quality_raw_2[i] = rxMsgData[i+4];
1374
1375         }
1376
1377         lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1378         quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1379
1380         //
1381         // Getting to this point means that the RX interrupt occurred on
1382         // message object 2, and the message RX is complete.  Clear the
1383         // message object interrupt.
1384         //
1385         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1386
1387         //
1388         // Since the message was received, clear any error flags.
1389         //
1390         errorFlag = 0;
1391     }
1392
1393
1394
1395     //
1396     // If something unexpected caused the interrupt, this would handle it.
1397     //
1398     else
1399     {
1400         //
1401         // Spurious interrupt handling can go here.
1402         //
1403     }
1404
1405     //
1406     // Clear the global interrupt flag for the CAN interrupt line
1407     //
1408     CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1409
1410     //
1411     // Acknowledge this interrupt located in group 9
1412     //
1413     InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1414 }
1415 // ----- code for CAN end here -----
```